

Exercises of the Day 12

1. Sliding Window + Hashing – Longest Substring Without Repeating Characters

Requirement: Sliding Window + Hashing check

Efficiency: $O(n)$ time, $O(\min(n, \text{charset size}))$ space

Problem:

Given a string s , find the length of the longest substring that contains no repeating characters.

Method to implement:

```
public static int lengthOfLongestSubstring(String s)
```

Driver Code:

```
import java.util.*;
```

```
public class LongestUniqueSubstring {
    public static int lengthOfLongestSubstring(String s) {
        // TODO: implement
        return 0;
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {"abcabcbb", 3},
            {"bbbbbb", 1},
            {"pwwkew", 3},
            {"", 0},
            {"a", 1},
            {"abcdef", 6},
            // large test to ensure  $O(n)$  (length  $10^5$ )
            {generateLargeString(100000), 26}
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            String input = (String) tests[i][0];
            int expected = (int) tests[i][1];
            long start = System.currentTimeMillis();
            int result = lengthOfLongestSubstring(input);
            long elapsed = System.currentTimeMillis() - start;
            if (result == expected && elapsed < 2000) {
                passed++;
                System.out.println("Test " + (i+1) + " passed (" + elapsed + " ms)");
            } else {
                System.out.println("Test " + (i+1) + " FAILED (got " + result + ", expected
```

```

    }
}
System.out.println(passed + "/" + tests.length + " tests passed");
}

private static String generateLargeString(int length) {
    StringBuilder sb = new StringBuilder(length);
    Random r = new Random(42);
    for (int i = 0; i < length; i++) {
        sb.append((char) ('a' + r.nextInt(26))); // only a-z, max unique = 26
    }
    return sb.toString();
}
}

```

2. Prefix Sum Validation – Range Sum Queries

Requirement: Prefix Sum validation

Efficiency: Preprocessing $O(n)$, each query $O(1)$ time; $O(n)$ extra space for prefix array

Problem:

Implement a class NumArray that is initialized with an integer array nums.

It has a method `int sumRange(int left, int right)` that returns the sum of elements from index left to right (inclusive). The driver will call `sumRange` many times.

Method to implement:

```

class NumArray {
    public NumArray(int[] nums) { }
    public int sumRange(int left, int right) { return 0; }
}

```

Driver Code:

```

import java.util.*;

public class RangeSumQuery {
    public static void main(String[] args) {
        int[] largeNums = new int[100000];
        Random r = new Random(123);
        for (int i = 0; i < largeNums.length; i++) largeNums[i] = r.nextInt(100) - 50;
        NumArray na = new NumArray(largeNums);

        // Test some random queries
        int[][] testQueries = {

```

```

        {0, 0},
        {0, largeNums.length-1},
        {100, 200},
        {99999, 99999}
    };
    // compute expected sums using linear scan (just for checking)
    int[] expectedSums = new int[testQueries.length];
    for (int q = 0; q < testQueries.length; q++) {
        int l = testQueries[q][0], r = testQueries[q][1];
        int sum = 0;
        for (int i = l; i <= r; i++) sum += largeNums[i];
        expectedSums[q] = sum;
    }

    boolean allPass = true;
    long start = System.currentTimeMillis();
    for (int q = 0; q < testQueries.length; q++) {
        int result = na.sumRange(testQueries[q][0], testQueries[q][1]);
        if (result != expectedSums[q]) {
            System.out.println("Query [" + testQueries[q][0] + ", " + testQueries[q][1] + "] FAIL");
            allPass = false;
        }
    }
    long elapsed = System.currentTimeMillis() - start;
    if (allPass && elapsed < 50) {
        System.out.println("All tests passed in " + elapsed + " ms");
    } else if (!allPass) {
        System.out.println("Some tests failed");
    } else {
        System.out.println("Too slow (took " + elapsed + " ms, expected << 100 ms for 0");
    }
}
}

```

3. Sorting Verification – Check if Array Is Sorted

Requirement: Check if candidate sorted correctly

Efficiency: $O(n)$ time, $O(1)$ space

Problem:

Write a method `isSorted(int[] arr)` that returns `true` if the array is sorted in non-decreasing order, `false` otherwise. An empty array or a single element is considered sorted.

Method to implement:

```
public static boolean isSorted(int[] arr)
```

Driver Code:

```
public class ArraySortedCheck {
    public static boolean isSorted(int[] arr) {
        // TODO: implement
        return false;
    }

    public static void main(String[] args) {
        int[][] tests = {
            {1,2,3,4,5},          // true
            {5,4,3,2,1},          // false
            {},                   // true
            {1},                  // true
            {1,1,1,1},            // true
            {2,1,3},              // false
            new int[100000]       // all zeros -> true (test efficiency)
        };
        // Fill the large array with zeros (already zero)
        boolean[] expected = {true, false, true, true, true, false, true};
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            long start = System.nanoTime();
            boolean result = isSorted(tests[i]);
            long elapsed = System.nanoTime() - start;
            if (result == expected[i] && elapsed < 1_000_000) { // less than 1ms for large
                passed++;
                System.out.println("Test " + (i+1) + " passed");
            } else {
                System.out.println("Test " + (i+1) + " FAILED (got " + result + ", expected
            }
        }
        System.out.println(passed + "/" + tests.length + " tests passed");
    }
}
```

4. Detect Inefficient $O(n^2)$ – Count Zero-Sum Subarrays ($O(n)$ via Prefix Sum + HashMap)

Requirement: Detect inefficient $O(n^2)$ solutions (by requiring $O(n)$ algorithm)
 Efficiency: $O(n)$ time, $O(n)$ space

Problem:

Given an integer array `nums`, return the number of contiguous subarrays whose

sum is zero.

Example: [1, -1, 1, -1] → subarrays: [1,-1] at (0,1), [1,-1,1,-1] at (0,3), [-1,1] at (1,2), [-1,1,-1,1]? etc. The correct answer is 4. Use a prefix sum + HashMap to achieve O(n).

Method to implement:

```
public static int countZeroSumSubarrays(int[] nums)
```

Driver Code:

```
import java.util.*;
```

```
public class ZeroSumSubarrays {
```

```
    public static int countZeroSumSubarrays(int[] nums) {
```

```
        // TODO: implement
```

```
        return 0;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Object[] [] tests = {
```

```
            {new int[] {1, -1, 1, -1}, 4},
```

```
            {new int[] {0, 0, 0}, 6},
```

```
            {new int[] {1, 2, 3}, 0},
```

```
            {new int[] {}, 0},
```

```
            {generateLargeArray(100000), 50000} // expected value depends on construction;
```

```
        };
```

```
        // For the large test, we'll generate an array with many zero sums: e.g., alternating
```

```
        // Let's set a known expected: for alternating 1, -1, the number of zero-sum subarrays
```

```
        // Simpler: use array of all zeros, then count = n*(n+1)/2.
```

```
        // So large array: 100000 zeros => 100000*100001/2 = 5000050000 (too large for int).
```

```
        // Actually we can adjust: I'll change to 10000 zeros, expected = 50005000 fits in
```

```
        // I'll replace the large case.
```

```
        // Let's replace last test manually:
```

```
        int[] largeZeros = new int[10000];
```

```
        int expectedLarge = 10000 * 10001 / 2;
```

```
        Object[] [] tests2 = {
```

```
            {new int[] {1, -1, 1, -1}, 4},
```

```
            {new int[] {0, 0, 0}, 6},
```

```
            {new int[] {1, 2, 3}, 0},
```

```
            {new int[] {}, 0},
```

```
            {largeZeros, expectedLarge}
```

```
        };
```

```
        int passed = 0;
```

```
        for (int i = 0; i < tests2.length; i++) {
```

```
            int[] input = (int[]) tests2[i][0];
```

```
            int expected = (int) tests2[i][1];
```

```
            long start = System.currentTimeMillis();
```

```

        int result = countZeroSumSubarrays(input);
        long elapsed = System.currentTimeMillis() - start;
        if (result == expected && elapsed < 2000) {
            passed++;
            System.out.println("Test " + (i+1) + " passed (" + elapsed + " ms)");
        } else {
            System.out.println("Test " + (i+1) + " FAILED (got " + result + ", expected " + expected + ")");
        }
    }
    System.out.println(passed + "/" + tests2.length + " tests passed");
}
}

```

5. Detect Infinite Loops – Floyd's Cycle Detection (Tortoise & Hare)

Requirement: Detect infinite loops / timeout mechanism

Efficiency: $O(n)$ time, $O(1)$ space (n = cycle length)

Problem:

You are given a function $f(x)$ (defined as `int f(int x)`) and a starting integer x_0 . Implement a method that returns `true` if the sequence $x_0, f(x_0), f(f(x_0)), \dots$ eventually repeats (i.e., contains a cycle). Use Floyd's cycle-finding algorithm.

Method to implement:

```
public static boolean hasCycle(int start, IntUnaryOperator f)
```

Driver Code:

```
import java.util.function.IntUnaryOperator;
```

```
public class CycleDetection {
```

```
    public static boolean hasCycle(int start, IntUnaryOperator f) {
        // TODO: implement
        return false;
    }
}
```

```
public static void main(String[] args) {
```

```
    // Test 1:  $f(x) = (x + 1) \% 5$ , start 0 => cycle 0->1->2->3->4->0...
```

```
    IntUnaryOperator f1 = x -> (x + 1) % 5;
```

```
    boolean exp1 = true;
```

```
    // Test 2:  $f(x) = x + 1$  (no cycle, but integer overflow may eventually loop? ignore).
```

```
    // For practical, use  $f(x) = \text{if } x < 100 \text{ then } x+1 \text{ else } x$ , so after 100, constant, i.e., no cycle.
```

```
    IntUnaryOperator f2 = x -> x < 100 ? x + 1 : x;
```

```
    boolean exp2 = true; // after 100, 100 -> 100 cycle
```

```
    // Test 3:  $f(x) = x + 1$  (no cycle) - but we need to limit, otherwise will never stop.
```

```

// For a truly non-cycling sequence, we'd need infinite range, but can't test. So w
// Simpler: define f that after some steps becomes constant (self-loop). That's a c
// Test 3: f(x) = if x == 10 then 5 else x+1; but this might have a cycle or not. W
// Let's do: no-cycle test with a function that always increases: x -> x+1 (but star
// I'll include a true no-cycle case by capping steps with a counter? The exercise i
Object[] [] tests = {
    {0, f1, true},
    {0, x -> x < 10 ? x + 1 : x, true}, // after 10, 10->10
    {0, x -> x == 5 ? 0 : x + 1, true} // 0,1,2,3,4,5->0 cycle
};
int passed = 0;
for (int i = 0; i < tests.length; i++) {
    int start = (int) tests[i][0];
    IntUnaryOperator f = (IntUnaryOperator) tests[i][1];
    boolean expected = (boolean) tests[i][2];
    boolean result = hasCycle(start, f);
    if (result == expected) {
        passed++;
        System.out.println("Test " + (i+1) + " passed");
    } else {
        System.out.println("Test " + (i+1) + " FAILED (got " + result + ", expected
    }
}
System.out.println(passed + "/" + tests.length + " tests passed");
}
}

```

6. Candidate Solution Analysis – Maximum Subarray Sum (Kadane's Algorithm $O(n)$)

Requirement: Compare to optimal approaches ($O(n)$ vs $O(n^2)$)

Efficiency: $O(n)$ time, $O(1)$ space

Problem:

Find the maximum sum of any contiguous subarray in a given integer array `nums`. Implement the classic Kadane's algorithm. The driver will test with a large array (size 10^5) that would timeout on an $O(n^2)$ solution.

Method to implement:

```
public static int maxSubArray(int[] nums)
```

Driver Code:

```
import java.util.*;
```

```
public class MaxSubarray {
```

```

public static int maxSubArray(int[] nums) {
    // TODO: implement
    return 0;
}

public static void main(String[] args) {
    Object[] [] tests = {
        {new int[]{-2,1,-3,4,-1,2,1,-5,4}, 6},
        {new int[]{1}, 1},
        {new int[]{5,4,-1,7,8}, 23},
        {new int[] {}, 0}, // convention: empty subarray sum = 0? We'll define as 0.
        {generateLargeArray(100000), 1} // will compute expected using Kadane internal
    };
    // pre-compute expected for large array: all values 1 -> max sum = length.
    int largeLen = 100000;
    int[] largeArr = new int[largeLen];
    Arrays.fill(largeArr, 1);
    tests[4] = new Object[]{largeArr, largeLen};

    int passed = 0;
    for (int i = 0; i < tests.length; i++) {
        int[] nums = (int[]) tests[i][0];
        int expected = (int) tests[i][1];
        long start = System.currentTimeMillis();
        int result = maxSubArray(nums);
        long elapsed = System.currentTimeMillis() - start;
        if (result == expected && elapsed < 2000) {
            passed++;
            System.out.println("Test " + (i+1) + " passed (" + elapsed + " ms)");
        } else {
            System.out.println("Test " + (i+1) + " FAILED (got " + result + ", expected " + expected + ")");
        }
    }
    System.out.println(passed + "/" + tests.length + " tests passed");
}

private static int[] generateLargeArray(int size) {
    int[] arr = new int[size];
    Arrays.fill(arr, 1);
    return arr;
}
}

```


7. Visual Feedback for Sliding Window – Moving Average (with Window Steps)

Requirement: Show window movement step-by-step (visual feedback)

Efficiency: $O(n)$ time, $O(n)$ output list (if all averages returned)

Problem:

Compute the moving average of a data stream. Write a method that takes an array `nums` and a window size `k`. Return a list of doubles representing the average of each sliding window of size `k` as it moves from left to right.

(Optionally, the driver prints each window to simulate visual feedback.)

Method to implement:

```
public static List<Double> movingAverage(int[] nums, int k)
```

Driver Code:

```
import java.util.*;
```

```
public class MovingAverage {
```

```
    public static List<Double> movingAverage(int[] nums, int k) {
```

```
        // TODO: implement
```

```
        return null;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Object[][] tests = {
```

```
            {new int[]{1,2,3,4,5}, 3, Arrays.asList(2.0, 3.0, 4.0)},
```

```
            {new int[]{5,5,5,5}, 1, Arrays.asList(5.0,5.0,5.0,5.0)},
```

```
            {new int[]{}, 3, Collections.emptyList()},
```

```
            {new int[]{10}, 1, Arrays.asList(10.0)},
```

```
            {new int[]{1,2,3,4,5}, 5, Arrays.asList(3.0)},
```

```
            {new int[]{1,2,3,4,5}, 6, Collections.emptyList()} // window larger than array
```

```
        };
```

```
        int passed = 0;
```

```
        for (int i = 0; i < tests.length; i++) {
```

```
            int[] nums = (int[]) tests[i][0];
```

```
            int k = (int) tests[i][1];
```

```
            List<Double> expected = (List<Double>) tests[i][2];
```

```
            List<Double> result = movingAverage(nums, k);
```

```
            if (expected.equals(result)) {
```

```
                passed++;
```

```
                System.out.println("Test " + (i+1) + " passed");
```

```
            } else {
```

```
                System.out.println("Test " + (i+1) + " FAILED");
```

```
                System.out.println("Input: " + Arrays.toString(nums) + " k=" + k);
```

```
                System.out.println("Expected: " + expected);
```

```
                System.out.println("Got: " + result);
```

```

    }
}
System.out.println(passed + "/" + tests.length + " tests passed");
}
}

```

8. Check for Duplicates – Contains Duplicate (HashSet)

Requirement: Check for duplicates in solution – hashing presence check

Efficiency: $O(n)$ time, $O(n)$ space

Problem:

Given an array of integers, return true if any value appears at least twice, false if every element is distinct.

Method to implement:

```
public static boolean containsDuplicate(int[] nums)
```

Driver Code:

```
import java.util.*;

public class ContainsDuplicate {
    public static boolean containsDuplicate(int[] nums) {
        // TODO: implement
        return false;
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {new int[]{1,2,3,1}, true},
            {new int[]{1,2,3,4}, false},
            {new int[] {}, false},
            {new int[]{1}, false},
            {new int[]{0,0,0,0}, true},
            // large test (10^5 distinct numbers -> false) to check efficiency
            {generateDistinctArray(100000), false}
        };
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            int[] nums = (int[]) tests[i][0];
            boolean expected = (boolean) tests[i][1];
            long start = System.currentTimeMillis();
            boolean result = containsDuplicate(nums);
            long elapsed = System.currentTimeMillis() - start;
            if (result == expected && elapsed < 500) {

```

```

        passed++;
        System.out.println("Test " + (i+1) + " passed (" + elapsed + " ms)");
    } else {
        System.out.println("Test " + (i+1) + " FAILED (got " + result + ", expected
    }
}
System.out.println(passed + "/" + tests.length + " tests passed");
}

private static int[] generateDistinctArray(int size) {
    int[] arr = new int[size];
    for (int i = 0; i < size; i++) arr[i] = i;
    return arr;
}
}

```

9. Validate Binary Search – Binary Search Implementation (Sorted Input Check)

Requirement: Validate binary search solutions – check sorted input requirement

Efficiency: $O(\log n)$ time, $O(1)$ space

Problem:

Implement binary search on a sorted array (ascending). Return the index of the target if found, otherwise return -1.

(The driver will provide only sorted arrays; you must still ensure your algorithm works correctly.)

Method to implement:

```
public static int binarySearch(int[] nums, int target)
```

Driver Code:

```

public class BinarySearchValidator {
    public static int binarySearch(int[] nums, int target) {
        // TODO: implement
        return -1;
    }

    public static void main(String[] args) {
        int[] sortedArray = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
        Object[] [] tests = {
            {sortedArray, 5, 4},
            {sortedArray, 1, 0},
            {sortedArray, 10, 9},
            {sortedArray, 11, -1},
            {new int[] {}, 5, -1},
        }
    }
}

```

```

        {new int[]{1}, 1, 0},
        {new int[]{1}, 2, -1}
    };
    int passed = 0;
    for (int i = 0; i < tests.length; i++) {
        int[] nums = (int[]) tests[i][0];
        int target = (int) tests[i][1];
        int expected = (int) tests[i][2];
        int result = binarySearch(nums, target);
        if (result == expected) {
            passed++;
            System.out.println("Test " + (i+1) + " passed");
        } else {
            System.out.println("Test " + (i+1) + " FAILED (got " + result + ", expected " + expected + ")");
        }
    }
    System.out.println(passed + "/" + tests.length + " tests passed");
}
}

```

10. Store Problem Test Cases – HashMap of Test Cases per Problem

Requirement: HashMap of test cases per problem

Efficiency: HashMap operations $O(1)$ average

Problem:

Implement a class `TestCaseManager` that stores and retrieves test cases for different problems.

- void `addTestCase(String problemId, String input, String expectedOutput)`
 – adds a test case. - `List<TestCase> getTestCases(String problemId)` –
 returns all test cases for that problem (as a list of `TestCase` objects). - `TestCase`
 is a simple class with two fields: `String input` and `String expectedOutput`.

Method to implement:

Complete the `TestCaseManager` class.

Driver Code:

```

import java.util.*;

class TestCase {
    String input;
    String expectedOutput;
    TestCase(String in, String out) { input = in; expectedOutput = out; }
    @Override
    public boolean equals(Object o) {

```

```

        if (!(o instanceof TestCase)) return false;
        TestCase t = (TestCase) o;
        return Objects.equals(input, t.input) && Objects.equals(expectedOutput, t.expectedOutput);
    }
}

public class TestCaseStore {
    static class TestCaseManager {
        // TODO: implement storage (e.g., Map<String, List<TestCase>>)
        public void addTestCase(String problemId, String input, String expectedOutput) {}
        public List<TestCase> getTestCases(String problemId) { return null; }
    }

    public static void main(String[] args) {
        TestCaseManager manager = new TestCaseManager();
        // Add test cases for two problems
        manager.addTestCase("sum", "1 2 3", "6");
        manager.addTestCase("sum", "0 0", "0");
        manager.addTestCase("max", "5 3 9", "9");
        manager.addTestCase("max", "1", "1");

        List<TestCase> sumTests = manager.getTestCases("sum");
        List<TestCase> expectedSum = Arrays.asList(
            new TestCase("1 2 3", "6"),
            new TestCase("0 0", "0")
        );
        List<TestCase> maxTests = manager.getTestCases("max");
        List<TestCase> expectedMax = Arrays.asList(
            new TestCase("5 3 9", "9"),
            new TestCase("1", "1")
        );

        boolean test1 = sumTests.equals(expectedSum);
        boolean test2 = maxTests.equals(expectedMax);
        // test unknown problem returns empty list
        boolean test3 = manager.getTestCases("unknown").isEmpty();

        System.out.println("Test 1: " + (test1 ? "passed" : "FAILED"));
        System.out.println("Test 2: " + (test2 ? "passed" : "FAILED"));
        System.out.println("Test 3: " + (test3 ? "passed" : "FAILED"));
    }
}

```

11. Score Based on Complexity – Analyze Loop Pattern (Simple Complexity Estimator)

Requirement: Score based on complexity – analyze loops/recursion patterns

Efficiency: Not about performance, but the analysis tool itself.

Problem:

Implement a function `scoreComplexity(String code)` that returns an integer complexity score based on the presence of nested loops in a simplified pseudo-code.

The pseudo-code is a single line containing the word `for` multiple times separated by spaces.

- Single `for` → score 1 ($O(n)$)
- Nested `for` (two consecutive `for for`) → score 2 ($O(n^2)$)
- Three nested → score 3, etc.
- If no `for`, score 0.

Example: "for for for" → 3, "for" → 1, "no loop" → 0.

Assume the code string only contains the word `for` and spaces; just count the longest consecutive sequence of `for`s.

Method to implement:

```
public static int scoreComplexity(String code)
```

Driver Code:

```
public class ComplexityScore {
    public static int scoreComplexity(String code) {
        // TODO: implement
        return 0;
    }

    public static void main(String[] args) {
        Object[][] tests = {
            {"for for for", 3},
            {"for", 1},
            {"for for", 2},
            {"", 0},
            {"no loop", 0},
            {"for something for", 2}, // "for for" adjacent after removing non-for words?
            {"for if for", 1}        // not consecutive
        };
        // Simple: split by spaces and find max consecutive "for" tokens
        int passed = 0;
        for (int i = 0; i < tests.length; i++) {
            String code = (String) tests[i][0];
            int expected = (int) tests[i][1];
            int result = scoreComplexity(code);
```

```

        if (result == expected) {
            passed++;
            System.out.println("Test " + (i+1) + " passed");
        } else {
            System.out.println("Test " + (i+1) + " FAILED (got " + result + ", expected
        }
    }
    System.out.println(passed + "/" + tests.length + " tests passed");
}
}

```

Each exercise targets a specific Day12 requirement and includes efficiency checks (via large inputs or explicit timeouts) where appropriate. Implement the empty methods, run the driver, and you'll see a clear pass/fail report.